

```
if faces>0:
```

```
    ser=serial.Serial('/dev/ttyUSB0',9600)
    print(ser)
    ser.write('Y')
```

```
else:
```

```
    ser=serial.Serial('/dev/ttyUSB0',9600)
    print(ser)
    ser.write('N')
```

I will not explain the OpenCV basics again here—please refer to Part 2 of this series. This code is quite similar to the *Face.py* code in that article. Basically, I have now imported the *pyserial* module, which will trigger Arduino. When we perform face detection, all detected contours are stored in the ‘*faces*’ variable (this means a non-zero and non-negative value if the operation is successful). So we do a basic condition check, and if successful, trigger Arduino and send a letter ‘Y’ to the serial port—else send a letter ‘N’.

Now let us look at the Arduino code (*BuildingES.pde*) below, which will respond to the above OpenCV code:

```
int ledPin = 13; // LED connected to digital pin 13

char incomingBit; // for incoming serial data

void setup() {
    pinMode(ledPin, OUTPUT); // sets the digital pin as
    output
    Serial.begin(9600); // opens serial port, sets data rate
    to 9600 bps
}
void loop() {
    if (Serial.available() > 0) {
        // read the incoming byte:
        incomingBit = Serial.read();
        // say what you got:
        //Serial.print("I received: ");
        Serial.println(incomingBit);

        if(incomingBit == 'Y' || incomingBit == 'y')

            digitalWrite(ledPin, HIGH); // sets
the LED off

            delay(1000);
        }
        else {
            digitalWrite(ledPin, LOW); // sets the
LED OFF
        }
    }
}
```

In the *if* condition in *loop()*, we used *Serial.available()*, which detects if any data has been sent to the serial port. If true, then *Serial.read()* will read the incoming character and store it in ‘*incomingBit*’. Then we can print what we have received from the serial port. After that, if the value of ‘*incomingBit*’ is ‘Y’, we need to turn the LED ‘on’, else we turn it ‘off’. So, in a nutshell, if detecting faces from a given image was successful, this will turn on the LED.

Why are we doing all this?

I am sure a lot of you will still have questions. Why are we doing all this? Where is the embedded system I was talking about? Well, let us have a quick recap of what we have actually done. We first used image processing to do some task. Then, if that task was successful, we triggered Arduino and made it turn on an LED. Now think of the bigger picture. The LED can be replaced by any electronic appliance in your house, like a fan, refrigerator or TV. (Obviously you will need a relay to control these appliances.) Look at this from the security point of view. We can design an embedded system that uses OpenCV for image-processing to detect an intruder, and trigger an alarm via the Pyserial module and Arduino. This is just one example. There are many awesome things that an embedded vision system (a mixture of an image-processing and an embedded system) can accomplish! We used Python here as that made it very easy to import different modules and exchange data or information between those modules. 

Links

- [1] <http://pyserial.sourceforge.net/index.html>
- [2] <http://www.arduino.cc/>

Acknowledgement

I would like to thank my colleagues Sohil Patel and Samartha Shah for their help and support. I would also like to thank Prof N P Gajar and Prof Dhaval Shah for their guidance.

By: Jayneil Dalal

The author is a FOSS advocate and loves to explore different open source technologies. His areas of interest include OpenCV, Python, Android, Linux, Arduino, BeagleBoard and other open source hardware platforms. In his spare time he likes to make video tutorials on various different open source technologies, which can be found on YouTube. He is a big Arsenal fan. He can be reached at jayneil.dalal@gmail.com.